

StegoVault: Deep Learning Execution Manual

Step-by-Step Operator Guide for Encoding and Decoding

1. Introduction to Deep Learning Steganography

StegoVault coordinates deep learning encoders to embed hidden text payloads within benign looking cover files. Unlike old LSB substitution techniques which alter pixels sequentially (creating statistical signatures), StegoVault uses convolutional neural nets and transformers to calculate optimal, distributed, and imperceptible modifications. This manual walks you through executing the deep learning pipeline for images, audios, videos, and text.

2. Stego-Encoding Workflow (Hiding Data)

Follow these steps to run the deep learning encoder:

- Step 1: Access the Dashboard.** Open your browser and navigate to `http://localhost:5173`. Verify that the 'AI Engine Health' indicator is green and showing active CPU or CUDA processors.
- Step 2: Choose Modality.** Under the Neural Modalities Hub (located on the right sidebar of the Dashboard), click the 'Encode' button for your target carrier file type (Image, Audio, Video, or Text).
- Step 3: Upload Cover File.** Drag and drop or upload a clean host file. Ensure it complies with the specific modality format limits (e.g. PNG for images, WAV for audio).
- Step 4: Enter Key & Payload.** Input a strong alphanumeric access key (used to salt and encrypt the message) and write your secret payload message in the text input area.
- Step 5: Generate Stego Carrier.** Click the 'Generate Stego Carrier' button. The frontend submits a POST request to `/api/encode/submit`, prompting the PyTorch neural network to execute a forward pass, embedding the data instantly.

3. Modality Selection Cheat-Sheet

Modality	Input Format	Output Format	Embedded Method
Image	PNG, JPG, BMP, WEBP	PNG (Lossless)	LSB Convolution (2D CNN)
Audio	WAV, FLAC	WAV (16-bit PCM)	Frequency Shift (1D CNN)
Video	MP4, AVI, MKV	MP4 (Lossless)	Spatiotemporal (3D CNN)
Text	TXT, PDF	TXT (Unicode)	Synonym Sharding (Transformer)

4. Stego-Decoding Workflow (Extracting Data)

Follow these steps to extract and decrypt your hidden message:

Step 1: Open Decryption Screen. Click the 'Decoding' option from the left sidebar navigation, or locate your file in the 'Secure Vault' list and click 'Decrypt'.

Step 2: Upload Stego File. Select the encoded stego container file (the output generated from the encoding workflow).

Step 3: Provide Security Key. Type in the exact access key that was entered when encoding. If the key doesn't match, the decryption logic will fail immediately due to AES-GCM tag mismatch.

Step 4: Extract Payload. Click the 'Extract Secret Payload' button. The backend reads the steganographic container footer, extracts the AES block, validates it, and displays the hidden text payload.

5. Advanced Options & Telemetry Management

During execution, you can view transaction analytics directly in the dashboard and vault. In the Secure Vault (Manage Vault Assets), clicking 'Details' opens the Asset Telemetry panel. This panel presents the SHA-256 bound file checksums, the device processor type (CUDA/CPU) used to run PyTorch, the raw AES key, and direct shortcuts to run the extraction process on that asset. If you encounter a processing bottleneck, verify that the 'AI Engine Health' panel latency metric is stable (usually below 150ms on CUDA and below 1200ms on CPU fallback).

6. Operator Rules for High Quality Results

To guarantee that your hidden message remains imperceptible to steganalysis tools, always follow these optimization constraints:

- 1. Never Compress Output Carriers:** Uploading stego images to messaging sites that automatically compress or crop images (like WhatsApp or Discord) will corrupt the high-frequency pixel changes. Always transmit carriers as raw, uncompressed documents.
- 2. Maintain FP32 Precision:** When executing python CLI scripts directly, ensure precision is set to Full 32-bit floats. Do not use float16 mixed precision unless memory space is limited, as rounding errors can introduce visual block noise.
- 3. Ensure CUDA is Loaded:** Always verify CUDA is active on the dashboard. CPU fallback uses slower float calculation threads, which increases latency and jitter metrics.

7. Troubleshooting Execution Errors

If a job returns a 'failed' status, check the terminal logs located at the bottom of the Secure Vault. The vault logs output a real-time trace of the SQLite write logs and the PyTorch error reports. Common failures include 'Modality limit exceeded' (trying to upload a carrier larger than 50MB) and 'Access key validation failed' (typing an incorrect password during decryption).